

Vision Transformers (ViTs)

Use self-attention mechanisms

Used in computer vision applications

Treat images as patch sequences

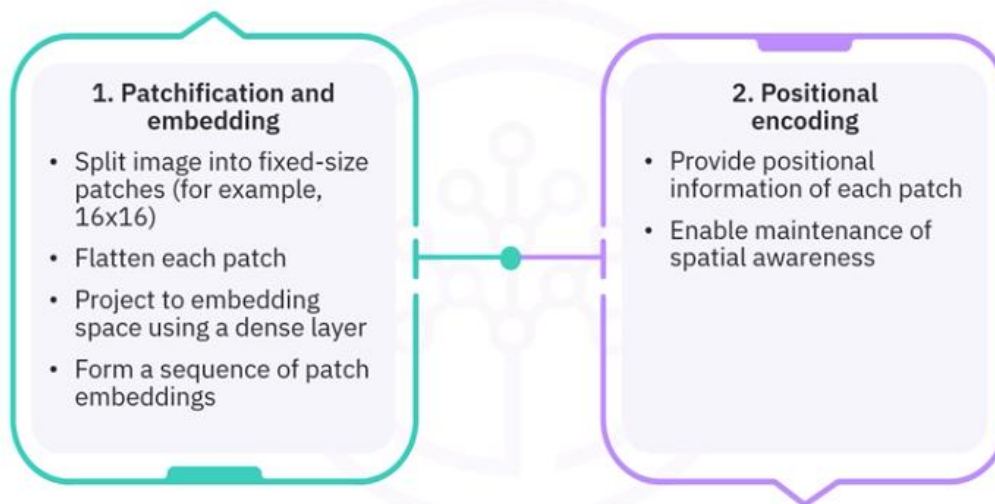
Excel with large datasets and computational resources

Hybrid model

- Combine CNNs + ViT
- Effective on small datasets

How ViTs Work

- Patchification and embedding → positional encoding → pass through transformer encoder blocks → classification



3. Passing through transformer encoder blocks

Each encoder block:

- Consists of multi-head self-attention
- Allows the model to weigh the relative importance of patches
- Captures local and global dependencies

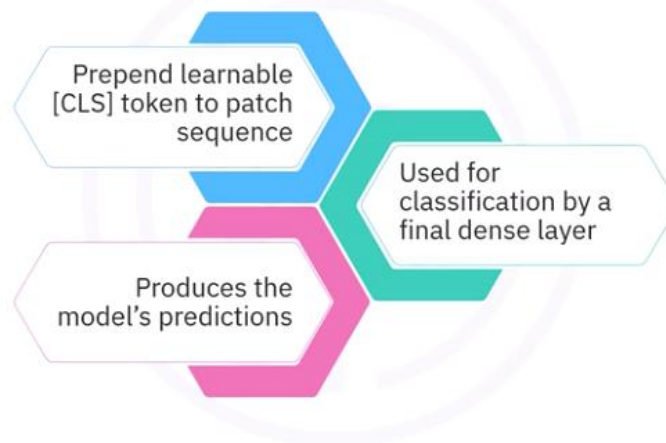
Feed-forward network:

- Applies a small neural network on each embedding

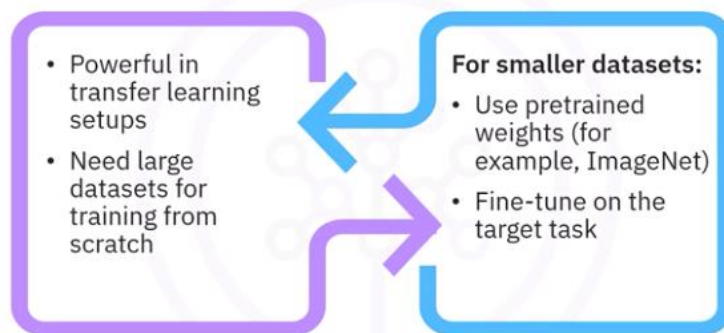
Residual connections and layer normalization:

- Stabilize training

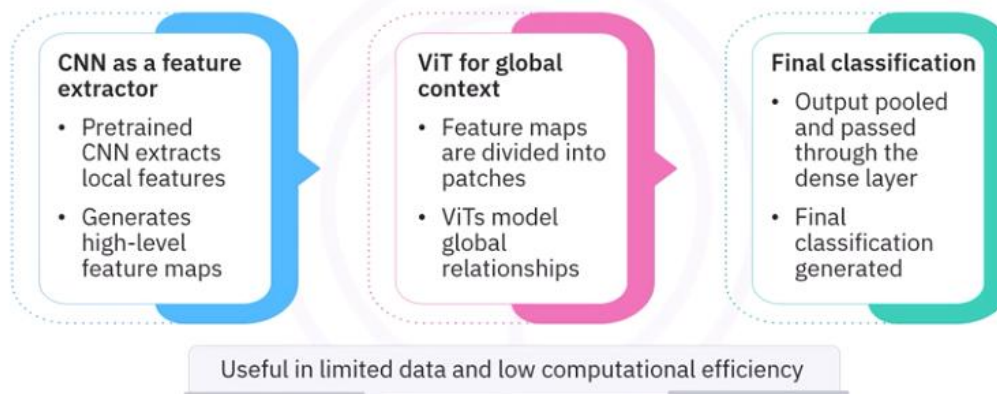
4. Classification



Vision Transformers and Transfer Learning



Combining CNNs and ViTs: Hybrid architecture*



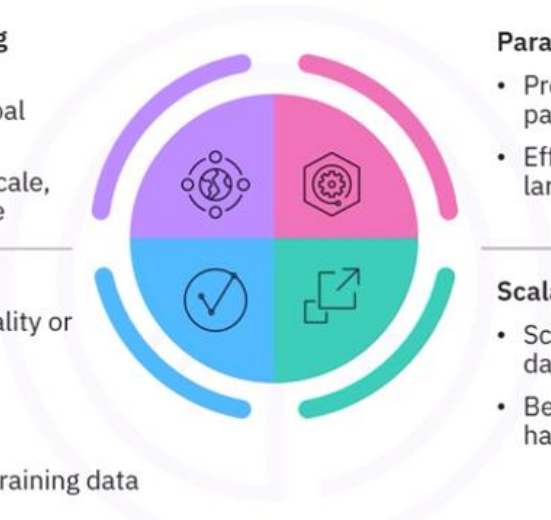
Mechanistic Details

Global context modelling

- Capture long-range dependencies and global relationships
- Robust to changes in scale, rotation, or perspective

Minimal inductive bias

- No assumptions of locality or translation invariance
- High flexibility
- Require more data
- Output dependent on training data



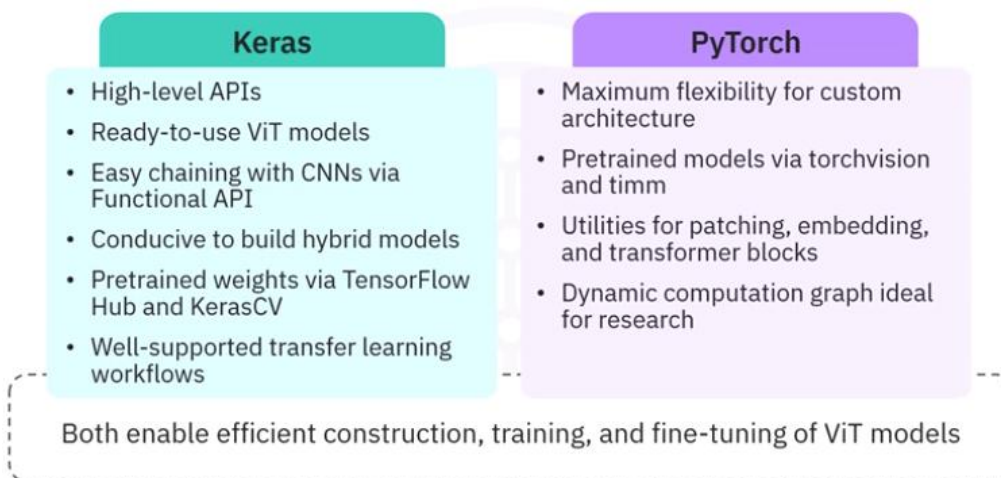
Parallel processing

- Process all patches in parallel
- Efficient computation for large data sets

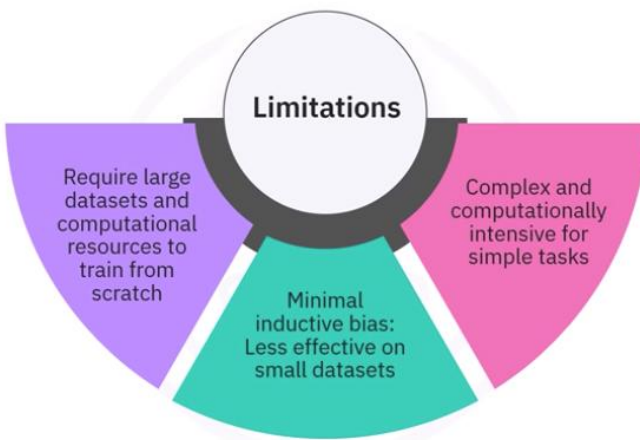
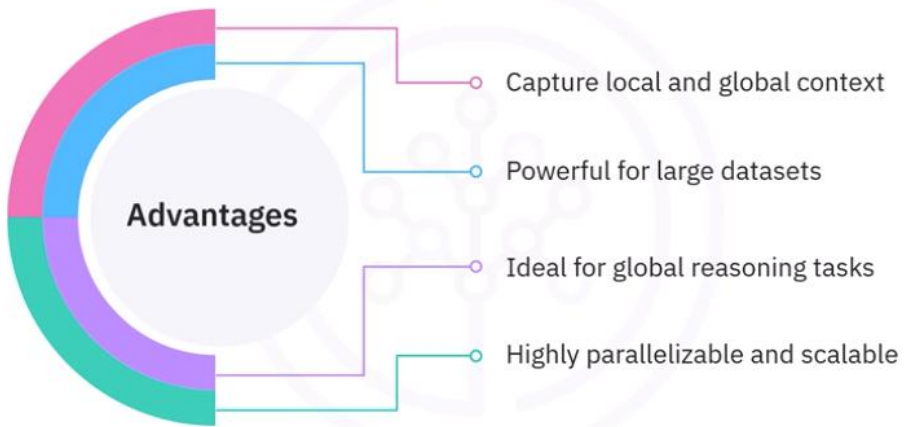
Scalability

- Scale to large models and datasets
- Benefit from advances in hardware and distributed training

ViTs in Keras and Pytorch



Advantages and Limitations of ViTs



At InnovateAI, a team of AI engineers led by Sarah is tasked with developing an image classification model using Keras. They decide to leverage vision transformers and transfer learning. What should be their initial step in implementing this model?

Collect a large dataset to train the vision transformer from scratch

Design a custom vision transformer architecture from scratch

Load a pre-trained vision transformer model and prepare the dataset for transfer learning

Implement a convolutional neural network as a baseline before using vision transformers

Starting with a pre-trained model allows the team to leverage existing knowledge and adapt it to their specific dataset, which is a key aspect of transfer learning.

Imagine that at AI Solutions Corp., the team led by Jamie is tasked with summarizing the use of vision transformers in Keras. What aspect should they emphasize to demonstrate the application of transfer learning in computer vision tasks?

The necessity of large datasets to train vision transformers effectively

The ability to adapt pre-trained models for specific tasks with minimal data

The requirement for extensive hyperparameter tuning in every application

The exclusive use of convolutional layers in vision transformers

This is a core advantage of transfer learning, allowing models to be tailored to new tasks efficiently.

At TechVision Inc., Alex is implementing a vision transformer model using PyTorch for a domain-specific image dataset. Which key step should Alex focus on to effectively apply transfer learning?

Use the model without any modifications on the new dataset

Train the model from scratch with the new dataset

Implement additional layers to the existing model

Fine-tune the pre-trained model on the new dataset

Fine-tuning involves adjusting the pre-trained model to better fit the new dataset, which is a crucial step in transfer learning.

What is the function of the self.features module inside the ConvNet class?

It loads the dataset into the model.

It computes the loss for the model during training.

It applies the pre-trained model architecture for CNN-based feature extraction.

It implements the transformer encoder for the hybrid architecture.

In the ConvNet definition, the self.features module defines the architecture of the pre-trained CNN model to process and extract hierarchical features from images using pre-trained model weights.

What is the primary operation performed by the PatchEmbed class's proj layer in the Vision Transformer implementation?

Projects the input feature map to a lower-dimensional embedding using a 1×1 convolution

Applies a pooling operation to flatten patches

Divides the image into overlapping patches using a 3×3 convolution

Flattens the original image directly into a sequence

The PatchEmbed class employs a 1×1 convolution via self.proj to convert feature maps into embeddings, serving as input tokens for the transformer.

When creating the positional encoding for image patches, which of the following operations is typically performed?

Adding a learned or fixed vector to each patch embedding

Subtracting the patch mean from each patch

Concatenating zeros to patch vectors

Reshuffling the patches randomly each epoch

Positional encoding involves adding a unique vector (learned or fixed) to each patch's embedding to retain positional information.

During Vision Transformer training, what is the typical purpose of including a "Classification Head" at the end of the model?

To increase model regularization

To project the final encoder output to the number of target classes

To encode positional information

To resize the image to its original shape

The classification head is a Dense layer that outputs logits or probabilities for each target class, enabling end-to-end training for classification tasks